Milestone 2 Report

Studying uGLAD: Implementation, Validation and a First Reproduction

CSE 516 - Probabilistic Graphical Models, Monsoon Semester 2026
Group: g12_th_conditional-ind-graphs_ind | Track: Theoretical AI (Individual)
Student: Het Patel, AU2440266

1. Problem Statement and Research Question

The problem is the same as in M1. I have samples of D variables. I want to know which pairs of variables stay connected after the effect of all the other variables is removed. For Gaussian data this means finding which entries of the precision matrix Θ are not zero. The base paper is uGLAD [1]. It learns this graph with a small neural network and does not need a ground truth graph during training.

In M1 I did not fix a research question, because my reading was not finished. For the project the question is now this:

How does the graph recovered by uGLAD change with the number of samples and with the training settings, compared with graphical lasso on exactly the same data?

I split this question over two milestones. In M2 I look at the first half in the setting of the paper itself. I use the data of the paper's Table 1 and the training settings that the authors' code uses by default. In M3 I will study other training settings, larger sample sizes and more baselines. I keep them out of M2 on purpose, so that each milestone has its own results.

The assumptions are the same as in M1. The data is Gaussian. The true precision matrix is sparse. The samples are independent and identically distributed.

In M2 I built and validated the full pipeline in code. I then ran a first experiment that follows the paper's Table 1 with the authors' own code, and compared it with my own implementation. All results in this report are preliminary.

2. Existing Work and SOTA Anchor (Revisited)

The review table in my M1 report is still correct. I did not add new papers to it. The only new references in M2 are the tools and algorithms that I used (Section 7).

My SOTA anchor is still Causal Order [7] from Microsoft Research. I kept it for its evaluation language, which is SHD, edge precision, edge recall and F1. I did not run it. It needs an imperfect expert such as a language model, and my setting has no such expert. So it is an anchor for how to report results and not a baseline in my tables. Cond-FiP [8] works on directed causal graphs, so I also treat it as related work only.

3. Representation: The Graphical Model as Code

The model is a Gaussian graphical model, also called a pairwise Markov random field. I wrote it as a Python class in ggm.py. The table below maps each idea to the code.

Table 1. The graphical model and where it lives in the code

Idea	In the model	In the code (ggm.py)
Variables	X1 to XD, jointly Gaussian with mean 0	GaussianGraphicalModel(precision)
Parameters	The precision matrix Θ, which is the inverse of the covariance	.theta
Nodes	One node for each variable	.d and .names
Edges	Edge i - j exactly when Θ_ij is not zero	.adjacency, .edges, .neighbors(i)
Edge strength	Partial correlation P_ij = -Θ_ij / sqrt(Θ_ii Θ_jj)	.partial_correlation
Node factor	exp(-0.5 Θ_ii x_i²)	.node_log_factor(i, x)
Edge factor	exp(-Θ_ij x_i x_j)	.edge_log_factor(i, j, x, y)
Joint distribution	p(x) is proportional to the product of all node and edge factors	.log_unnormalized(x) and .log_density(x)

A zero in Θ means that two variables are conditionally independent given all the others. In the factor form, a zero means that the edge factor is missing, so the two variables never appear in the same factor. I checked this in code. The sum of the node and edge log factors differs from the exact log density only by one constant, with a spread of 5e-14 over 50 random points (check V6e).

A small example shows why Θ and not the covariance is used. Take three variables in a chain X1 - X2 - X3 with Θ = [[2, -1, 0], [-1, 2, -1], [0, -1, 2]]. The covariance $\Sigma = \Theta^{-1}$ has all entries nonzero, and X1 and X3 have a correlation of 1/3. But Θ13 = 0, so the partial correlation of X1 and X3 is exactly 0. They are linked only through X2. The partial correlation of X1 and X2 is 0.5.

This representation fits the problem for two reasons. It shows direct links between variables, which plain covariance cannot do. It also keeps inference cheap, which Section 5 explains.

4. Learning: Structure and Parameters

In a Gaussian model the structure and the parameters are one object. The nonzero pattern of Θ is the structure and the values of Θ are the parameters. Learning means estimating Θ from data. Then I read the edges from the nonzero entries.

uGLAD has a second kind of parameter. These are the weights of its small neural networks. They are not model parameters. They are parameters of the optimizer that finds Θ. I keep the two kinds apart on purpose. The model has D(D+1)/2 free numbers. The optimizer has 42 numbers, whatever the value of D (check V5c).

The objective

Both graphical lasso and uGLAD work on the same objective. Here S is the sample covariance and ρ is the sparsity penalty.

$$\text{minimize over } \Theta: \quad -\log\det\Theta + \operatorname{trace}(S\,\Theta) + \rho\,\Sigma_{i\neq j}\,|\Theta_{ij}|$$

Classical learning

I use block coordinate descent from scikit-learn (GraphicalLassoCV). It uses one penalty value for the whole matrix. The value is chosen by cross validation. This is the baseline called SGLCV in the paper.

uGLAD learning

uGLAD splits Θ into two copies, Θ and Z, and adds a coupling term with weight λ. It then alternates two steps. The Θ step has a closed form, built from a matrix square root: with $b = S/\lambda - Z$, the new Θ is ½(−b

+ sqrt($b^T b + 4I/\lambda$)). The Z step is a soft threshold of every entry. I unroll 15 of these alternations into a network. One layer does six things:

- Form the helper matrix $b = S/\lambda - Z$.
- Compute the new Θ in closed form.
- Collect three numbers for every entry: the new Θ value, the S value and the last Z value.
- A small network turns these three numbers into a threshold ρ_{ij} between 0 and 1.
- Soft threshold: $Z_{ij} = \text{sign}(\Theta_{ij}) \cdot \max(|\Theta_{ij}| - \rho_{ij}, 0)$. This creates exact zeros.
- A second small network gives the next coupling weight λ.

The same two small networks are used in all 15 layers. The output is Z after layer 15. The network is trained without any ground truth graph. The loss is $-\log\det Z + \text{trace}(S\,Z)$, which is the likelihood with no sparsity term. I train with Adam, learning rate 0.002, for 250 epochs, which are the defaults of the authors' code. I use 3-fold cross validation: I train on part of the rows, check the same loss on the held-out rows after every epoch and keep the best epoch. Before training the columns are scaled to [0, 1].

I wrote the square root step as my own PyTorch function. Its forward pass uses an eigendecomposition. Its backward pass divides by the sum of two square roots of eigenvalues. I checked it against finite differences (check V3b).

This learning method fits the problem for a clear reason. The objective is convex and its Θ step has a closed form. So the unrolled network always returns a positive definite matrix in the Θ steps, and it is small enough to train on a single dataset.

5. Inference

Inference means answering questions with the learned model. Because the model is Gaussian, every query is exact and has a closed form. I do not need message passing or sampling. The queries I implemented are below.

Table 2. Queries, how they are answered and what they cost

Query	Answer	Code
Are X_i and X_j independent given all others?	Yes exactly when $\Theta_{ij} = 0$	is_conditionally_independent
How strongly and in which direction are they linked?	Partial correlation, from Θ	partial_correlation
Predict X_i from all other variables	Mean = $-(\Sigma_j \Theta_{ij} x_j) / \Theta_{ii}$, variance = $1 / \Theta_{ii}$	predict_node
Distribution of a set A given a set B	Gaussian with mean and covariance from the precision of A and B; other variables are integrated out first	conditional
Marginal of a set of variables	Gaussian with the matching block of $\Sigma = \Theta^{-1}$	marginal

This fits the representation. The graph tells which variables matter for a query. I checked this directly. Predicting a variable from its graph neighbours only gives the same answer as predicting from all other variables, with a difference of 3e-16 (check V6f). I also checked every query against Monte Carlo samples (V6a to V6d).

The cost is one matrix inverse or one solve. I measured it. Predicting one variable takes about 13 microseconds at D = 25 and 56 microseconds at D = 400. All partial correlations at D = 400 take 1.2 milliseconds. These numbers are in Results/validation/inference_query_time.csv.

6. Mathematical Formulation and Validation

Before running any experiment I checked the maths and the code. There are 23 checks in 8 groups. All of them pass. The table gives one line for each group. The full list with the measured numbers is in Results/validation/validation_report.md.

Table 3. Validation checks

Group	What is checked	Result
V1	Partial correlation formula against the Schur complement of the covariance. Zero in Θ against zero partial correlation. Also in samples.	Max error 1.1e-16 (exact), 0.0048 (samples)
V2	The closed form Θ step solves the stationarity equation, stays positive definite and is a minimum.	Gradient norm 2.3e-13
V3	My matrix square root against scipy. Backward pass against finite differences.	Error 1.0e-14. Gradcheck passes
V4	The alternating steps never increase their objective. Their fixed point against an independent L-BFGS minimizer.	Largest increase 2.7e-15. Relative difference 1.0e-8
V5	uGLAD network: every Θ_k is positive definite, output is symmetric, 42 parameters for D = 10, 25 and 100, training lowers the loss.	Smallest eigenvalue 4.05. Loss -33.5 to -54.4
V6	Marginals, conditionals, factorization and the local Markov property against Monte Carlo and exact algebra.	Max error 0.0027 (samples), 5e-14 (algebra)
V7	Moralization of DAG and CPDAG outputs (for the PC, GES and hill climbing baselines planned for M3). Metrics on a hand worked example.	All as worked by hand
V8	My uGLAD against the public uGLAD code on 4 datasets. Extended to 60 datasets in Fig. 2.	Max AUPR difference 0.0002 on the 4 datasets

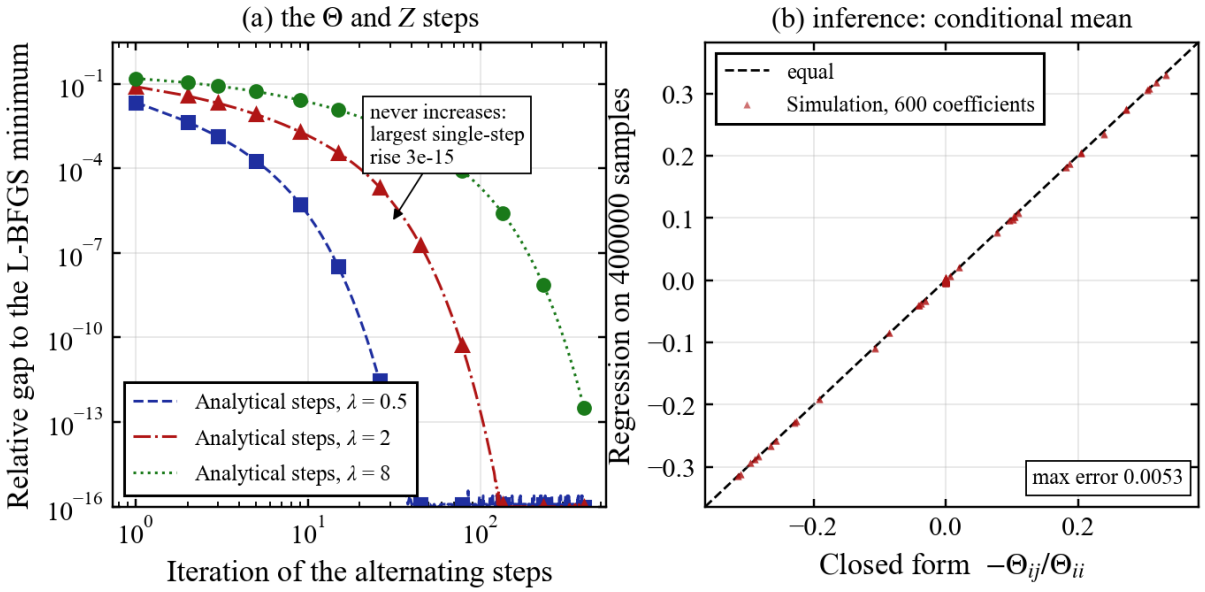


Fig. 1: The mathematics checked against independent references. (a) The alternating steps used inside uGLAD, with three coupling weights on a D = 8 problem: the objective never rises (largest single-step increase 3e-15) and reaches the minimum found by a generic optimizer (L-BFGS). (b) The closed-form coefficient -Θij/Θii of the conditional mean against the coefficient fitted by regression on 400000 simulated samples; the largest difference over 600 coefficients is 0.0053.

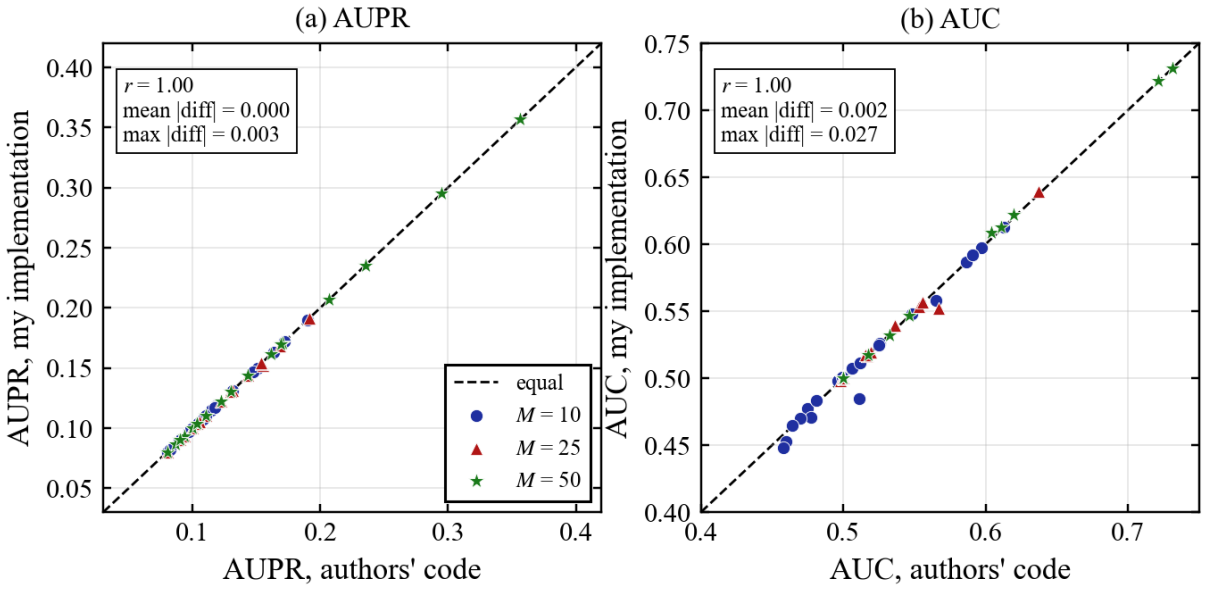


Fig. 2: My implementation against the authors' code on the same 60 datasets with the same seeds. Each marker is one dataset. The two programs rank the node pairs alike: the correlation of AUPR is 1.00 and the mean absolute difference is 0.000 (largest 0.003). They are close but not identical, because the matrix square root is computed differently.

Two honest comments. First, my implementation and the authors' code are not identical. The AUC differs by up to 0.027 on single datasets, because the authors compute the matrix square root with an iteration in single precision and I use an eigendecomposition in double precision. They agree on how they rank the node pairs (1.00 correlation of AUPR). Second, the classical alternating steps with fixed settings are slow at the scale of my experiments: after 15 steps the objective is still 67% away from its final value. So the 15 learned layers of uGLAD cannot be explained as a short run of the classical method with fixed settings.

7. Baseline and Benchmark Setup

Dataset

I use synthetic Gaussian data. I follow the protocol of Section 7.1 of the uGLAD paper [1] and I generate the data with the authors' own generator from their repository, with the paper's settings. First an Erdős-Rényi random graph [9] with edge probability p = 0.1. Then a full matrix of independent values from U(−1, 1). The entries on the edges are kept and the matrix is averaged with its transpose. A multiple of the identity is added so that the smallest eigenvalue is 1. This is the true precision matrix. Then I draw M samples from a zero mean Gaussian with that precision. I set D = 25, which gives about 30 true edges out of 300 pairs. The sample sizes are M = 10, 25 and 50, with 20 graphs for each, as in the paper. Graph k uses seed 2026 + k. For the learning and inference KPIs I draw 1000 fresh test samples from the true model. There is no real dataset in M2.

Baseline and methods

Table 4. Methods compared

Method	Type	Tool and settings
SGLCV (baseline)	Undirected, convex, one penalty chosen by CV	scikit-learn [10] GraphicalLassoCV with default settings, as in the paper
uGLAD, authors' code	Undirected, learned optimizer	Public package [16], CV mode, 250 epochs, learning rate 0.002, 3 folds
uGLAD, my code	Same design, my PyTorch code [11]	Same settings as the line above
True model (reference)	The precision matrix that made the data	Best value that any method can reach

All methods see the same samples. As in the public demo, the authors' uGLAD scales the columns to [0, 1] inside its fit function and SGLCV runs on the raw samples. The ranking scores AUC and AUPR use the absolute value of the learned entry of every node pair. For the likelihood and prediction KPIs, I convert all learned matrices back to the raw units of the data, so the methods are comparable. PC, GES and hill climbing are implemented in baselines.py (with the skeleton and moral graph conversions checked in V7). I run them in M3, not here.

KPIs

Table 5. KPIs

Layer	KPI	How it is computed
Graph	AUPR, AUC	Rank the 300 node pairs by $\lvert\hat{\Theta}_{ij}\rvert$. A random ranking gives an AUPR of about 0.10 (the share of true edges) and an AUC of 0.5.
Graph	Number of predicted edges	Entries with $\lvert\hat{\Theta}_{ij}\rvert$ above 1e-10. Also the number of empty graphs.
Learning	Held-out negative log likelihood	Per test sample on 1000 samples, in raw units. The true model gives the best possible value.
Learning	Parameters and run time	42 for the uGLAD network. Median seconds for each dataset.
Inference	Conditional prediction error	RMSE of the exact conditional mean of each variable given all the others, on the test samples.
Inference	Query time	Microseconds for one query (Section 5).

8. Initial Experiments and KPI Results

Every cell is the mean and the standard deviation over 20 graphs. No run failed. The numbers come from Results/reproduction/table1_author_generator/.

8.1 Edge recovery against the paper

Table 6. AUPR and AUC against Table 1 of the paper (D = 25, p = 0.1, 20 graphs)

Method	AUPR M=10	AUPR M=25	AUPR M=50	AUC M=10	AUC M=25	AUC M=50
SGLCV, paper	0.112 ± 0.013	0.132 ± 0.012	0.219 ± 0.085	0.505 ± 0.007	0.532 ± 0.031	0.617 ± 0.083
SGLCV, my run	0.123 ± 0.050	0.153 ± 0.048	0.240 ± 0.070	0.518 ± 0.041	0.540 ± 0.040	0.604 ± 0.054
uGLAD, paper	0.159 ± 0.029	0.174 ± 0.018	0.223 ± 0.062	0.572 ± 0.027	0.595 ± 0.044	0.651 ± 0.021
uGLAD, authors' code, my run	0.119 ± 0.031	0.122 ± 0.030	0.147 ± 0.075	0.518 ± 0.050	0.520 ± 0.035	0.544 ± 0.074
uGLAD, my code	0.119 ± 0.031	0.122 ± 0.030	0.147 ± 0.075	0.516 ± 0.051	0.520 ± 0.035	0.545 ± 0.075

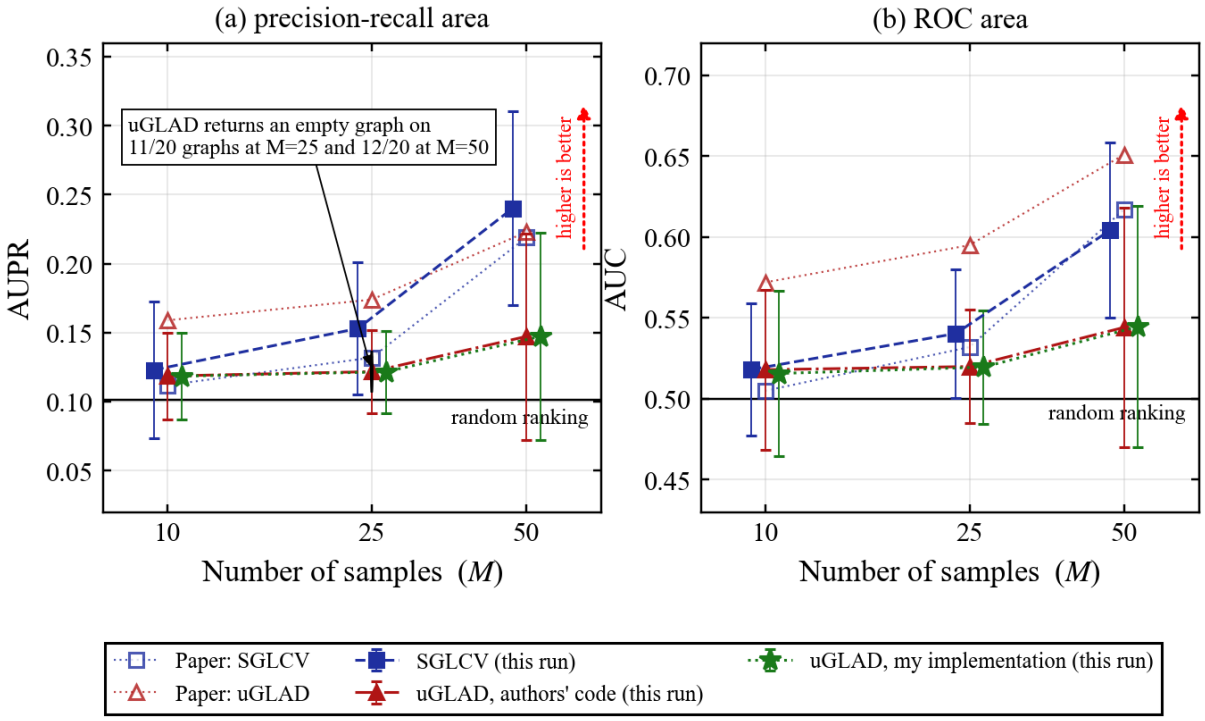


Fig. 3: Edge recovery against the number of samples, mean and one standard deviation over 20 graphs (D = 25, p = 0.1), with the values printed in the uGLAD paper as dotted hollow markers. The SGLCV curve reproduces the published one: the largest AUPR difference is 0.021, inside one standard deviation at every M. uGLAD run with the authors' code at its default settings does not: it is 0.040 to 0.076 below the published AUPR and, over the same graphs, differs from SGLCV by -0.004, -0.031 and -0.093 AUPR at M = 10, 25 and 50 (paired t = -0.5, -3.3, -4.9).

- The baseline is reproduced. My SGLCV agrees with the paper's SGLCV to within 0.021 AUPR at every M, which is inside one standard deviation.
- uGLAD with the authors' code at its default settings is below the published numbers. It is tied with SGLCV at M = 10 (AUPR difference -0.004, paired t = -0.5; on AUC -0.000). It is behind SGLCV at M = 25 (-0.031, t = -3.3) and at M = 50 (-0.093, t = -4.9).
- My own implementation gives the same numbers as the authors' code on the same data (Fig. 2). So this result belongs to the method at these settings and not to a bug in my code.

8.2 Graph by graph

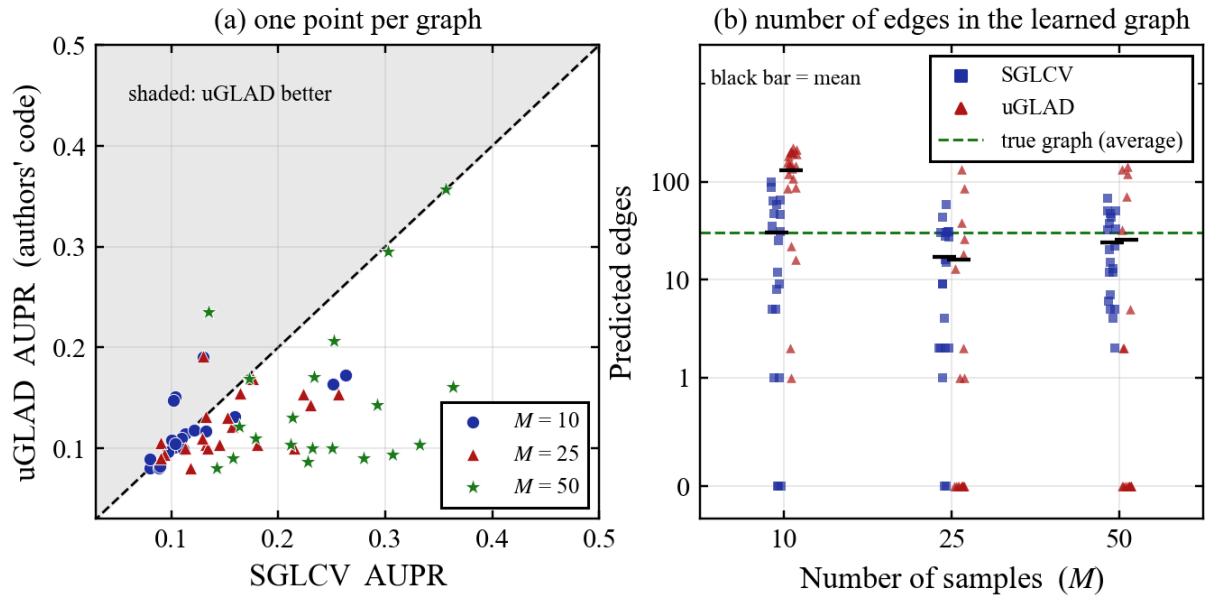


Fig. 4: (a) AUPR of uGLAD against AUPR of SGLCV on the same graph, so a point above the dashed line is a graph where uGLAD wins; uGLAD is above the line on 7, 2 and 1 of 20 graphs at M = 10, 25 and 50. (b) Number of edges in the learned graph, one marker per graph (vertical axis linear up to 1, logarithmic above). uGLAD returns an empty graph 11 and 12 times out of 20 at M = 25 and 50, and about 130 edges on average at M = 10, where the true graph has about 30.

At M = 10 uGLAD returns far too many edges, 130 on average where the truth has 30. At M = 25 and 50 it returns a graph with no edges at all on 11 and 12 of 20 datasets. SGLCV does this on 2 and 0. I have not yet studied why. Possible reasons are the training length and the threshold that the network learns. This is the first thing I will look at in M3.

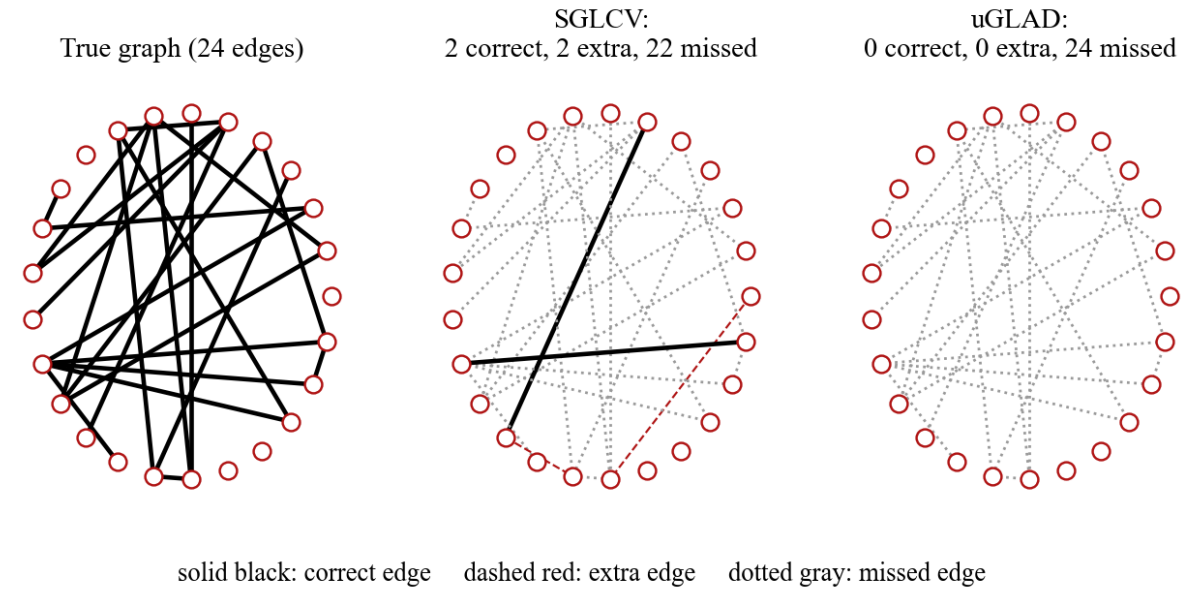


Fig. 5: One recovered graph. It is the first of the 20 graphs at the largest M (M = 50, graph index 0); the rule was fixed before looking, so it is not a picked example. The true graph has 24 edges. SGLCV finds 2 of them and adds 2; uGLAD finds 0 and adds 0.

8.3 Learning and inference KPIs

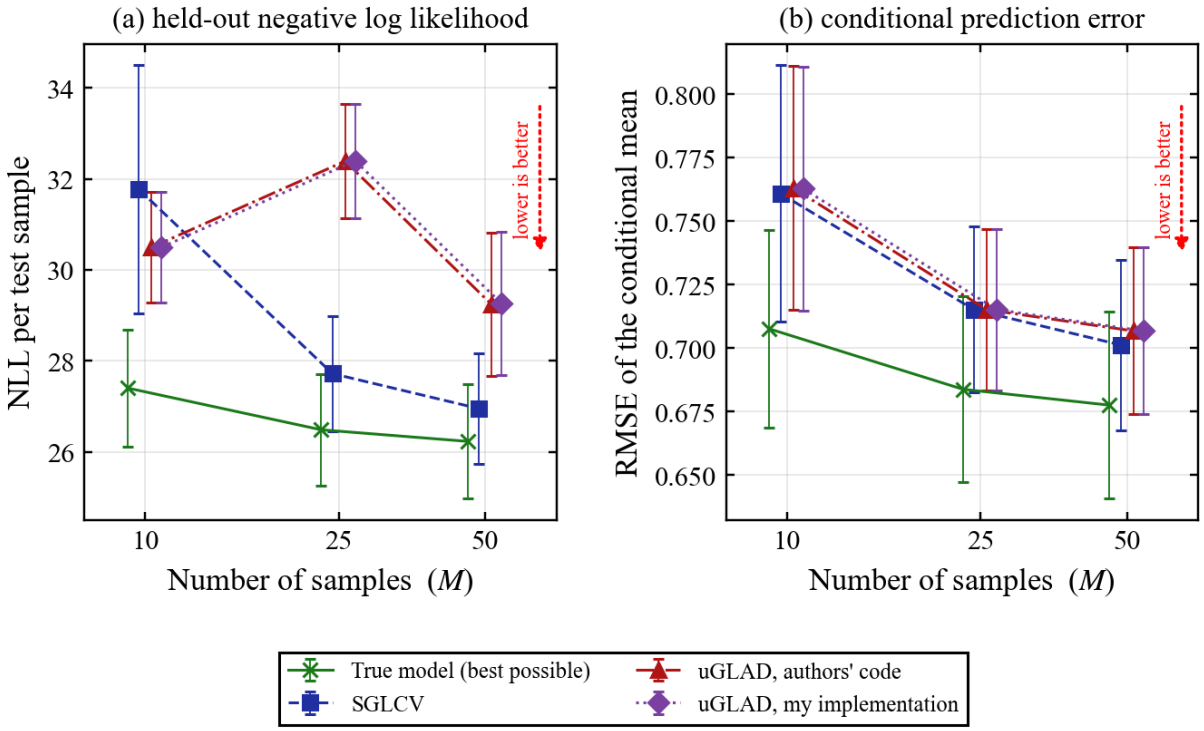


Fig. 6: Learning and inference KPIs on 20 test graphs per M, mean and one standard deviation, 1000 fresh test samples per graph, all methods compared in the raw units of the data. (a) Held-out negative log likelihood; the green curve is the true model, so no method can go below it. (b) Error of the exact conditional mean of each variable given all the others. At M = 50 the gap to the true model is 0.7 nats for SGLCV and 3.0 for uGLAD, but the prediction error differs by only 0.023 and 0.029.

Table 7. Held-out negative log likelihood and conditional prediction error (lower is better)

Method	NLL M=10	NLL M=25	NLL M=50	RMSE M=10	RMSE M=25	RMSE M=50
True model	27.40 ± 1.28	26.48 ± 1.23	26.23 ± 1.25	0.708 ± 0.039	0.684 ± 0.037	0.678 ± 0.037
SGLCV	31.77 ± 2.73	27.72 ± 1.26	26.95 ± 1.23	0.761 ± 0.051	0.715 ± 0.033	0.701 ± 0.034
uGLAD, authors' code	30.50 ± 1.22	32.39 ± 1.26	29.24 ± 1.58	0.763 ± 0.048	0.715 ± 0.032	0.707 ± 0.033
uGLAD, my code	30.50 ± 1.22	32.40 ± 1.26	29.26 ± 1.58	0.763 ± 0.048	0.715 ± 0.032	0.707 ± 0.033

The prediction error is within 0.02 to 0.05 of the true model for every method. So this KPI alone cannot tell a good graph from a poor one. The likelihood shows more: at M = 50 SGLCV is 0.7 below the best possible value and uGLAD is 3.0, which matches the empty graphs.

8.4 Size and run time

The uGLAD network has 42 learnable parameters at every D. One uGLAD fit took a median of 63 seconds with the authors' code and 34 seconds with mine, against about 1 second for SGLCV. The runs shared a laptop with other work, so I treat the times as rough.

9. A Mismatch With the Paper That I Found and Fixed

My first experiment used my own data generator. It drew one symmetric U(−1, 1) number for every edge. With that data my SGLCV scored a much higher AUPR than the paper's SGLCV. I compared my pipeline with the authors' code line by line and found one real difference. The authors' generator averages the weight matrix with its transpose, so every edge value is the mean of two uniform draws. The edges are weaker and the problem is harder. The paper does not mention this. Everything else was already the same: the model call and its defaults, the scaling, the baseline and the scoring functions. The audit is in Results/reproduction/VERIFICATION.md.

Table 8. SGLCV mean AUPR over 20 graphs with the two generators

	M = 10	M = 25	M = 50
Paper	0.112	0.132	0.219
SGLCV, my first generator	0.145	0.205	0.360
SGLCV, authors' generator (used in M2)	0.123	0.153	0.240

The first run is not used in M2. I kept its files for M3.

10. Limits

- I ran only the setting of the paper's Table 1: D = 25, p = 0.1 and M = 10, 25, 50, with 20 graphs for each and one series of seeds.
- I used the default settings of the authors' code (250 epochs, learning rate 0.002). The paper gives a learning rate range of 0.0001 to 0.001 and does not give the number of epochs. So I cannot say that the paper's exact training was matched.
- The paper does not say how SGLCV was run (folds, preprocessing). I used the scikit-learn defaults.
- All data is synthetic and Gaussian. I have not tested noise, missing values or real data.
- The run times were measured on a laptop that was shared with other work, so they are rough.

11. Revisiting M1

- Problem framing. In M1 I had only the problem as the paper poses it. Now I have a research question (Section 1), split over M2 and M3.

- A correction to M1. M1 said that the values ρ and λ are learned at each step. This can be read as separate values or separate networks for each step. In fact the same two small networks are used in all 15 layers. Section 4 describes it correctly.
- Reproducing the public code. M1 planned to reproduce the public uGLAD code. I did this for Table 1 (Section 8) and I also wrote my own implementation, so that I can explain each line. I use the public code as a cross check.
- Baselines. In M1 I said that PC, GES and hill climbing would need to be adapted to undirected graphs. The conversion code is written and tested (V7). The experiments with them are planned for M3.
- SOTA anchor. I kept Causal Order as the anchor (Section 2).

12. Reproducibility

All code, the saved datasets and the learned matrices, and the results are in the M2 folder. Code/README.md gives the setup, the exact package versions, the seeds and the run commands. The raw numbers are in Results/reproduction/table1_author_generator/. Every figure and every number in this report comes from the scripts in Code/src and nothing is typed in by hand. The live demo is demo.py. Running the authors' code needs a clone of their repository (see Code/README.md).

13. Plan for M3 and M4

Table 9. Plan (may change as I learn more)

Milestone	Planned work
M3	Study the training length and the learning rate of uGLAD. Larger sample sizes. The baselines PC, GES and hill climbing. A rule to turn the learned matrix into a graph. Noise and missing data. Significance tests over more graphs.
M4	Consolidate the results. Revisit the SOTA position. Write the final analysis.

References

[1] H. Shrivastava, U. Chajewska, R. Abraham, and X. Chen, “uGLAD: A Deep Learning Model to Recover Conditional Independence Graphs,” Microsoft Research, presented at the NeurIPS 2022 Workshop on New Frontiers in Graph Learning (GLFrontiers), 2022.

[2] H. Shrivastava, X. Chen, B. Chen, G. Lan, S. Aluru, H. Liu, and L. Song, “GLAD: Learning Sparse Graph Recovery,” in International Conference on Learning Representations (ICLR), 2020.

[3] J. Friedman, T. Hastie, and R. Tibshirani, “Sparse inverse covariance estimation with the graphical lasso,” Biostatistics, vol. 9, no. 3, pp. 432-441, 2008.

[4] H. Shrivastava and U. Chajewska, “Neural Graphical Models,” in Proceedings of the 17th European Conference on Symbolic and Quantitative Approaches to Reasoning with Uncertainty (ECSQARU), 2023.

[5] H. Shrivastava and U. Chajewska, “Neural Graph Revealers,” Workshop on Machine Learning for Multimodal Healthcare Data, International Conference on Machine Learning (ICML), 2023.

[6] H. Shrivastava and U. Chajewska, “Methods for Recovering Conditional Independence Graphs: A Survey,” arXiv:2211.06829, 2022.

[7] A. Vashishtha, A. G. Reddy, A. Kumar, S. Bachu, V. Balasubramanian, and A. Sharma, “Causal Order: The Key to Leveraging Imperfect Experts in Causal Inference,” in International Conference on Learning Representations (ICLR), 2025.

[8] D. Mahajan, J. Gladrow, A. Hilmkil, C. Zhang, and M. Scetbon, “Amortized Inference of Causal Models via Conditional Fixed-Point Iterations,” Transactions on Machine Learning Research (TMLR), 2025.

[9] P. Erdős and A. Rényi, “On random graphs I,” Publicationes Mathematicae Debrecen, vol. 6, pp. 290-297, 1959.

[10] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[11] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[12] Y. Zheng, B. Huang, W. Chen, J. Ramsey, M. Gong, R. Cai, S. Shimizu, P. Spirtes, and K. Zhang, “Causal-learn: Causal Discovery in Python,” Journal of Machine Learning Research, vol. 25, no. 60, pp. 1-8, 2024.

[13] A. Ankan and A. Panda, “pgmpy: Probabilistic Graphical Models using Python,” in Proceedings of the 14th Python in Science Conference (SciPy), pp. 6-11, 2015.

[14] D. M. Chickering, “Optimal Structure Identification With Greedy Search,” Journal of Machine Learning Research, vol. 3, pp. 507-554, 2002.

[15] P. Spirtes, C. Glymour, and R. Scheines, Causation, Prediction, and Search, 2nd ed. MIT Press, 2000.

[16] H. Shrivastava, uGLAD public code, version 2.0.0, github.com/Harshs27/uGLAD.